

# The Living Map — Mine/Tunnel Rescue

IEEE TSYP14 · AESS × RAS Technical Challenge — Phase 1 Technical Report

GitHub: [github.com/abderahmenamry-ai/Tsyp14](https://github.com/abderahmenamry-ai/Tsyp14)

## 1. Project Overview

In disaster response, robots exploring collapsed structures, mines or contaminated zones often operate with no GPS and no communication network. If a robot fails mid-mission, the knowledge it gathered is lost with it. The Living Map gives the space its own memory: a **Writer** robot autonomously explores and deposits small radio beacons at points of interest, and a separate **Executor** robot later retrieves that information through a dedicated **Outside Network Area** and continues the mission, without ever needing a direct link to the command post. Our environment is **Mine/Tunnel Rescue**, modelled on a Gafsa-style room-and-pillar phosphate mine with branching galleries, a dead-end chamber and a rock-fall-blocked branch. We detect three event types: toxic gas, structural collapse and trapped worker.

## 2. Technical Solution & Architecture

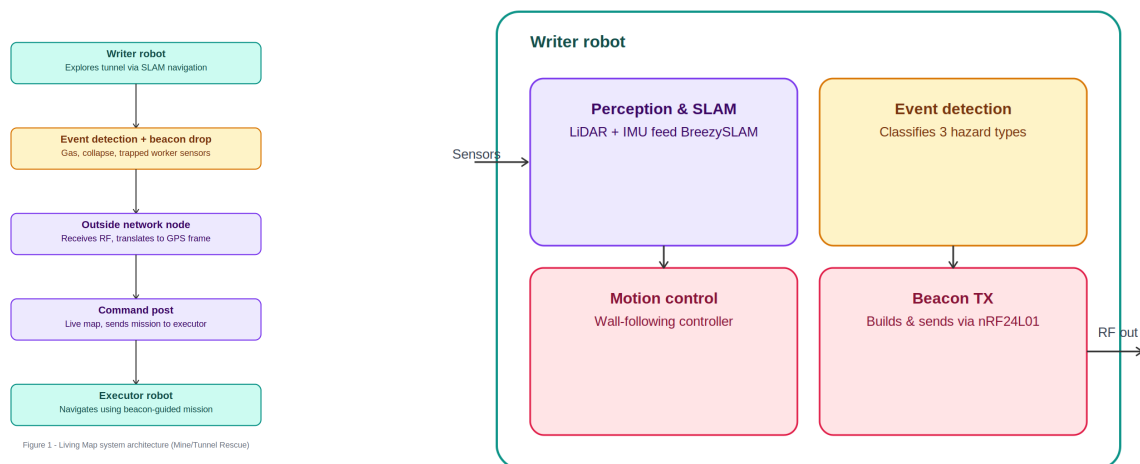


Fig. 1 - System: Writer → beacon → outside network → command post → Executor.

Fig. 2 - Writer robot internal subsystems.

### Writer robot autonomy

The Writer combines a simulated 2D LiDAR (360 rays, 8 m range, PyBullet ray casting) and a noisy IMU/odometry model with **BreezySLAM** (RMHC scan matching) to map and localise without GPS. Exploration uses a right-hand-rule wall-following controller with four states (FOLLOW, TURN\_LEFT, SEEK, RECOVER). A known SLAM failure mode, heading drift along long, feature-poor corridors ( $-0.2^\circ/\text{m}$ ), is mitigated with damped odometry/scan-match fusion and a wall-direction heading reference.

### Event detection & beacon deposition

Detection triggers when the robot's true position enters one of three predefined hazard zones. Each event is stamped with the SLAM-estimated position (used in the beacon) alongside the ground-truth position (kept for evaluation), which lets us quantify localisation error end to end. Beacon position error was 0.08 m (worker), 0.13 m (collapse) and 0.23 m (gas). **A beacon is a radio message, not a physical object:** the Writer transmits it and it is kept alive by retransmission and aging, which avoids a mechanical drop mechanism.

### Beacon message & signal design

A beacon is a compact 16-byte message (ID, event type, priority, local x/y, heading, sensor value, timestamp, TTL), well inside the 32-byte payload of a low-cost nRF24L01. Priority derives from the event type (trapped worker and collapse = high, gas = medium) and drives an **aging** mechanism: higher-priority beacons start with a longer TTL (low 150 s, medium 360 s, high 600 s) and are retried more often, so urgent findings persist and are retrieved first.

### Frame translation

The Outside Network node converts beacon local coordinates to GPS with a one-time manual calibration at the tunnel entrance (GPS position + compass bearing) and a flat-earth approximation valid at this scale. Manual calibration was chosen over an onboard magnetometer, since compasses are unreliable near a mine's metal structures and mineral content.

## Outside network area

All robot-command-post traffic passes through the Outside Network node; no direct link exists, which satisfies the challenge constraint. The node receives beacons, translates coordinates and forwards them to the command post, which aggregates data, renders a live map and builds the Executor's mission as a priority-sorted list of unvisited targets.

## Executor robot

The Executor consumes the prioritised list and navigates to each target with an A\* planner over its own SLAM occupancy map (walls inflated 0.45 m, unknown space passable but costlier), followed by pure-pursuit path tracking with periodic replanning. Across four tested random seeds it retrieved all 3 beacons.

## What is new

Findings live in the network as acknowledged, aging beacons rather than on the robot, so knowledge outlives the Writer. Priority-aware aging separates how long a beacon lives from how hard it is retried. The Writer verifies its weakest sensor by stopping to re-check a PIR trigger before declaring a trapped worker. The whole firmware was tested against simulated faults and a 50% lossy radio before any hardware existed.

# 3. Robot Hardware, Firmware & Communication

Each robot is an ESP32 differential-drive platform (TB6612FNG driver, quadrature-encoder gear motors, MPU-6050 gyro, LD06 360° LiDAR, nRF24L01+ radio). The Writer adds the three event sensors. The outside-network node is an ESP32 with a radio only. All three roles run **one C++ code base**, selected at build time (`-DFW_ROLE_WRITER / EXECUTOR / HUB`), so a fix to framing, odometry or the radio layer is made once.

## Firmware structure

A hardware-abstraction layer (*hal.h*) isolates everything the logic needs from the outside world. The portable core above it contains: host-link framing with CRC, the LD06 packet parser and scan assembly, wheel odometry with gyro heading, a per-wheel PID with acceleration limiting, the event detectors (MQ-2 sustained rise above baseline, tilt debounce with repeat window, PIR with warm-up lockout), the beacon sender (retries until acknowledged, TTL aging) and the Writer, Executor and Hub applications. A thin ESP32 layer (*hal\_esp32.cpp*, *main.cpp*) supplies pins, PWM, encoders, I2C, UART and SPI. Because the core is hardware-independent, the same code also compiles for a PC and runs inside the simulation.

## Host link and safety

The ESP32 handles all hard real-time work (motor control, sensing, radio). LiDAR scans, odometry and events stream over USB serial to a host (laptop now, Raspberry Pi later) that runs BreezySLAM and the navigation controllers and sends back velocity commands. The **safety layer lives on the ESP32, independent of the host**: it stops the robot if the host goes silent for 0.5 s, if the LiDAR stops delivering, or on an emergency-stop command.

## Radio protocol

Robots never talk to the command post directly. A Writer beacon is acknowledged by the hub; unacknowledged beacons are retried, and stale ones are dropped by TTL. The Executor asks the hub for its mission over the radio, receives targets, and reports each one reached reliably. The hub de-duplicates beacons, applies the GPS translation and hands the result to the command post over its own host link.

# 4. Verification Results (simulation and firmware)

Everything was run on a PC with no hardware. Four layers of testing were used:

| Layer                | What it checks                                                                                                                                                                                                                                 | Result                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| C++ unit tests       | Wire-format layout and CRC, framing, LD06 parser, odometry, wheel PID, event detectors, beacon retry and aging                                                                                                                                 | 57 checks, 0 failures |
| Behaviour tests      | Boot and gyro calibration, closed-loop ( $v, \omega$ ) $\rightarrow$ PID $\rightarrow$ encoders $\rightarrow$ odometry, safety layer, sensors $\rightarrow$ events, hub GPS translation and de-duplication, Writer/Executor over a lossy radio | 62 checks, 0 failures |
| ESP32 build          | Syntax check of the ESP32 glue for Writer, Executor and Hub                                                                                                                                                                                    | Pass (syntax check)   |
| Firmware-in-the-loop | Real firmware driving the PyBullet mine through the full Writer $\rightarrow$ hub $\rightarrow$ Executor mission, with perfect radio and with 50% packet loss                                                                                  | See below             |

|                                | Writer (Python sim)                                                                | Writer (real firmware in loop)                 |
|--------------------------------|------------------------------------------------------------------------------------|------------------------------------------------|
| Mission                        | 448 s, 146 m, returned to start                                                    | Completes mission                              |
| Position error vs ground truth | mean 0.14 m, max 0.28 m (odometry alone ends 4–8 m off)                            | mean 0.25 m, max 0.45 m                        |
| Wall contacts                  | 0                                                                                  | 0                                              |
| Beacons                        | 3 of 3 (0.08 / 0.13 / 0.23 m error)                                                | 3 acknowledged and delivered                   |
| Executor                       | 3 of 3 in priority order; 136 s, 63 m, max error 0.28 m (also 3 of 3 on seeds 1–3) | 3 of 3; 144 s, max error 0.26 m                |
| With 50% radio loss            | –                                                                                  | Same result: retries still deliver all beacons |

Running the real firmware raises the mean error from 0.14 m to 0.25 m. The firmware path adds wheel gain mismatch, motor lag and stiction, and gyro bias and noise that the idealised odometry model does not.

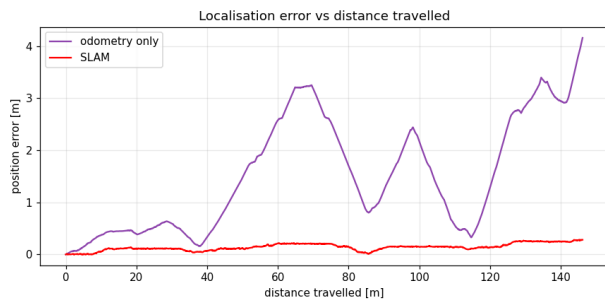


Fig. 3 – Position error: SLAM vs. odometry-only.



Fig. 4 – Command-post live map at the end of the mission.

## 5. Event Detection Sensors

The three event sensors use a plain analog or digital pin (no I2C/SPI), keeping wiring and debugging simple for a beginner team. Each is a low-cost, well-documented breakout module.

**Toxic gas — MQ-2.** Tin-dioxide semiconductor sensor that responds to LPG, propane, methane, hydrogen, alcohol and smoke over about 300–10,000 ppm. Analog output read through a 10k/15k divider (the ESP32 ADC is 3.3 V max). It is not selective for toxic gases and drifts with temperature and humidity, so the firmware uses a sustained-rise-above-baseline trigger, not a calibrated measurement.

**Structural collapse — SW-520D tilt/vibration switch.** Sealed rolling-ball switch (no mercury) giving a binary tilted/not-tilted state at about 14–45° of tilt. No calibration or library. Firmware debounces and requires repeat triggers in a short window, since it also trips on the robot's own bumps.

**Trapped worker — HC-SR501 PIR.** Detects a moving warm body in a cone up to about 7 m / 120°. It needs 30–60 s warm-up (locked out in firmware), is most reliable while the robot is stationary, and cannot detect a worker who is perfectly still.

## 6. Implementation Plan (Phase 2 Roadmap)

| Item                                                     | Qty    | Est. cost           | Notes                                                           |
|----------------------------------------------------------|--------|---------------------|-----------------------------------------------------------------|
| ESP32 dev board                                          | 3      | \$15–21             | Writer, Executor, outside-network node                          |
| Motor driver + gear motors with encoders + chassis       | 2 sets | \$26–41             | Encoders are required for SLAM odometry                         |
| MQ-2 / SW-520D / HC-SR501                                | 1 each | \$5–9               | Event sensors (Writer only)                                     |
| nRF24L01 radio                                           | 3      | \$4–8               | Writer, Executor, outside node                                  |
| Battery + regulation + wiring                            | –      | \$13–20             | 5 V motor/sensor rail, 3.3 V logic                              |
| <b>Base total</b>                                        |        | <b>≈ \$65–90</b>    | Matches implementation_plan.pdf                                 |
| Navigation sensing (A: ultrasonic + servo / B: 2D LiDAR) | 2 sets | \$26–40 / \$146–210 | + IMU each; Option B matches our simulation and firmware (LD06) |

**Build order:** (1) one robot driving, (2) LiDAR/IMU and SLAM on hardware, (3) event sensors tested individually, (4) radio link and beacon TX/RX test, (5) second robot and goal-seeking controller, (6) outside-network node, (7) full integration test. The firmware repository already contains a step-by-step **bring-up checklist** for the first power-on: radio only, motor and encoder polarity, geometry calibration, wheel-PID tuning, LiDAR CRC and orientation, sensor thresholds, safety layer, then radio range.

**Testing plan:** bench-test each sensor alone (gas baseline and rise, tilt debounce, PIR range with the robot stopped, motor/IMU straight-line and turn error), then a hallway or cardboard-maze navigation test with pose

error comparable to simulation, a radio range and packet-loss test at increasing distance, shared start-frame and calibration checks, and a full integration test in a mock tunnel with safe hazard stand-ins.

**Timeline (about 8 weeks):** wk 1-2 parts and robot #1 driving; wk 3-4 SLAM on hardware and sensor bench tests; wk 5 radio link and range tests; wk 6 robot #2 and outside node; wk 7 integration testing; wk 8 fixes, retest and demo recording. Main risks: part shipping delays, SLAM tuning with real sensor noise, and radio range underground. Week 8 is a spare buffer.

## 7. Failure Cases

---

**SLAM heading drift in long, featureless corridors.** Stock BreezySLAM drifts about  $-0.2^\circ/\text{m}$  in straight corridors (worse while turning:  $-36^\circ$  over 18 s). A perfectly featureless tunnel gives scan matching nothing to anchor to. Mitigation: damped scan-match/odometry/gyro fusion, no correction while rotating fast, and a wall-direction heading reference, reducing mean error to 0.14 m versus 4-8 m odometry-only. Remaining: the reference assumes an orthogonal mine layout; irregular tunnels need it disabled.

**Sensor false positives and negatives.** The MQ-2 is non-selective and drifts with temperature and humidity (mitigated by sustained-rise triggering). The PIR can trigger on warm moving air and misses a motionless worker, a risk the sensor choice cannot fully remove. The tilt switch trips on the robot's own bumps (mitigated by debounce and repeat windows). Simulation does not exercise these cases; Phase 2 bench tests will.

**RF range limits.** nRF24L01 range is cut further by rock and metal underground. We tested the protocol with 50% packet loss and the acknowledge-and-retry layer still delivered every beacon, but retries do not help once the Writer is fully out of range. Known directions, not built: a multi-node relay chain and PA+LNA radio variants if range tests demand it.

**Frame translation assumptions.** Flat-earth approximation with one-time manual calibration is accurate at single-tunnel-network scale; a  $2^\circ$  heading error shifts a beacon 100 m out by about 3.5 m. The Executor's driving is unaffected, since it uses beacon coordinates directly in the shared entrance frame.

**Executor navigation.** An earlier straight-line controller reached only 1 of 3 beacons; the current A\* + pure-pursuit controller reaches 3 of 3 across four seeds. Limits: unknown space is treated as passable (it can head into an unexplored dead end before replanning), steering gains are reused from the wall-follower, and the tunnel is assumed static.

**Firmware not yet run on hardware.** All results are simulation and PC-in-the-loop. The ESP32 images have not been run on real boards, so wheel slip, LiDAR CRC details (the LD06 polynomial is from the vendor description and unverified), battery voltage (not measured in this revision) and clock sync between nodes for beacon aging are open items for Phase 2.